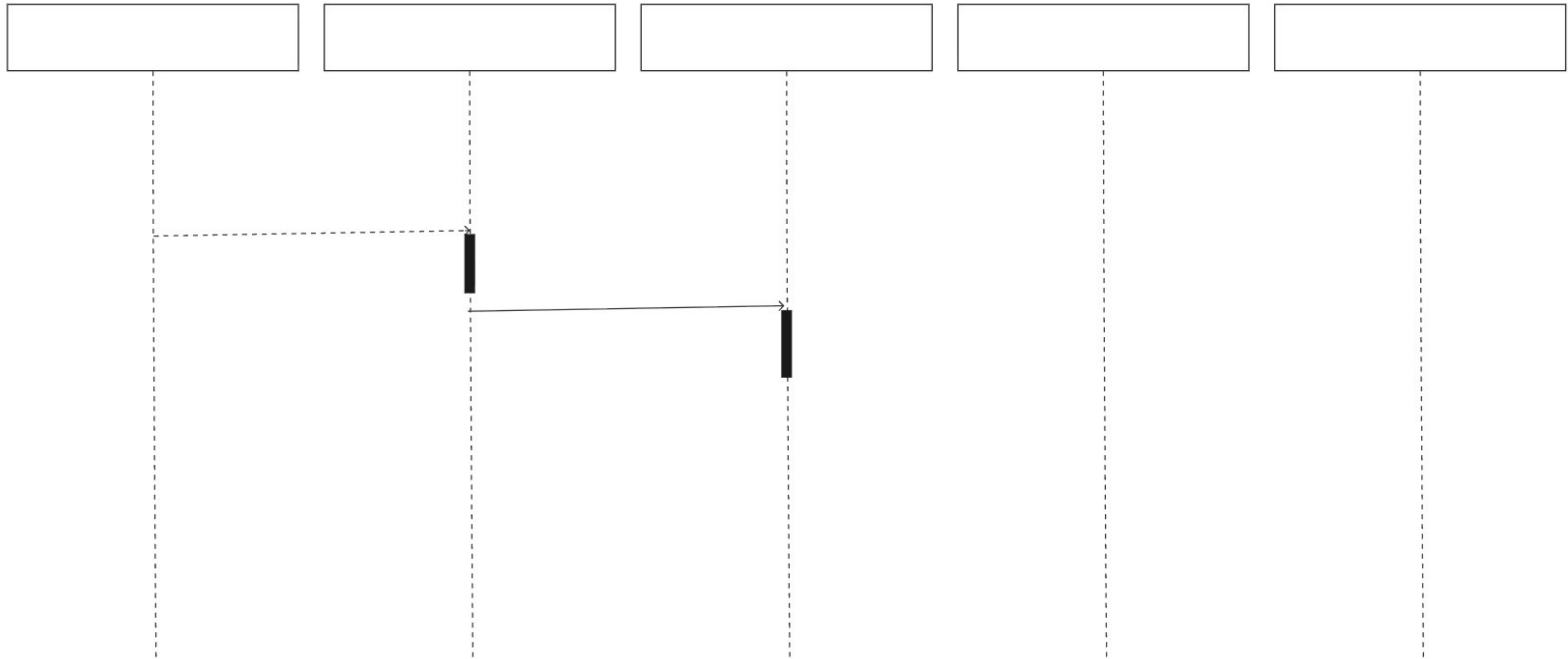


Sequence Diagram



What Is a Sequence Diagram, Really

A class diagram answers: what exists in the system.

A sequence diagram answers: what happens, in what order, when one specific action is triggered.

SSD v/s SD

There's a related but different diagram called a System Sequence Diagram (SSD). An SSD treats your entire system as one black box — one lifeline. You show the actor (user) and the system as a single participant exchanging messages. It's used early in design, before you've decided on internal components.

A full sequence diagram breaks that black box open. Instead of one "System" lifeline, you show the actual internal objects — controllers, managers, repositories — and how they talk to each other to fulfill that one use case.

Part 2 — The Core Building Blocks

2.1 Actor / Object (the top box)

Every participant — whether it's a human actor or a software object/class — is drawn as a box at the top of the diagram.

If it's a human: draw an actor stick figure or a plain box, labelled with the role (e.g., "Customer").

If it's a software object: draw a rectangle labelled objectName : ClassName or just the class name if there's only one instance that matters (e.g., BookingFacade).



2.2 Lifeline

From the bottom of each box, a vertical dashed line drops down. This is the lifeline.

Why dashed, not solid? The dashed line represents passive existence — the object is alive and available, but not necessarily doing anything at that moment. It's a timeline, not an action. Time flows top to bottom along this line.

2.3 Activation Bar (Focus of Control)

When an object is actively executing — processing a method, doing work — you draw a narrow solid rectangle on top of its lifeline, starting at the moment it receives a call and ending when it finishes and returns control.

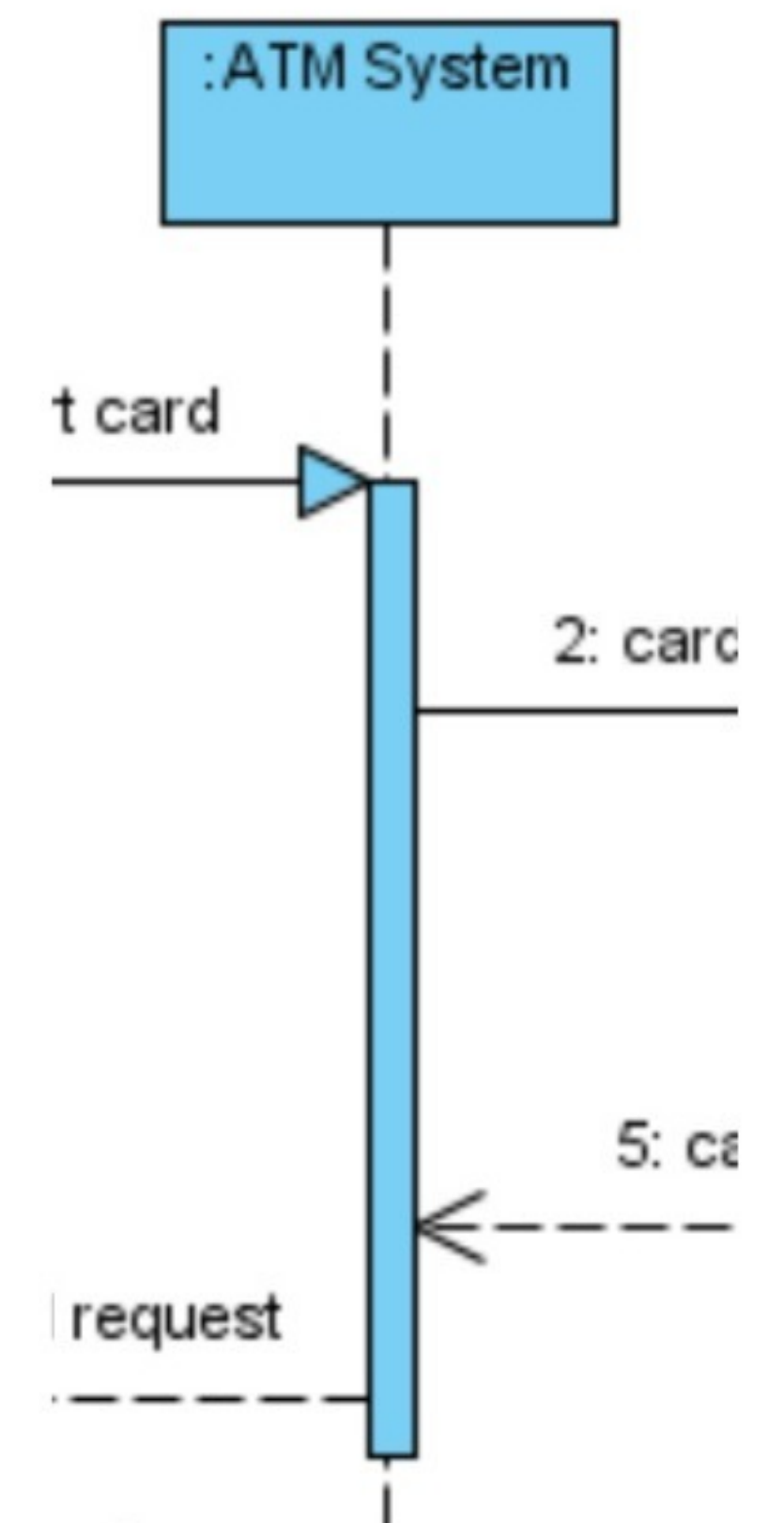
Why does this matter, and why should you draw it precisely?

It visually answers "who is doing work right now" at any point in time.

Nested activation bars (a bar starting inside another bar) show that an object called another object while it was still busy — which is exactly what happens in our BookMyShow flow: BookingFacade is active while it waits for SeatManager to respond.

An activation bar that never closes is a diagram bug — it means you forgot to show the return.

Rule: every solid call arrow that lands on a lifeline should start an activation bar. Every return arrow leaving that lifeline should end it.



Part 3 — Message Types (This Is the Part Interviewers Actually Test)



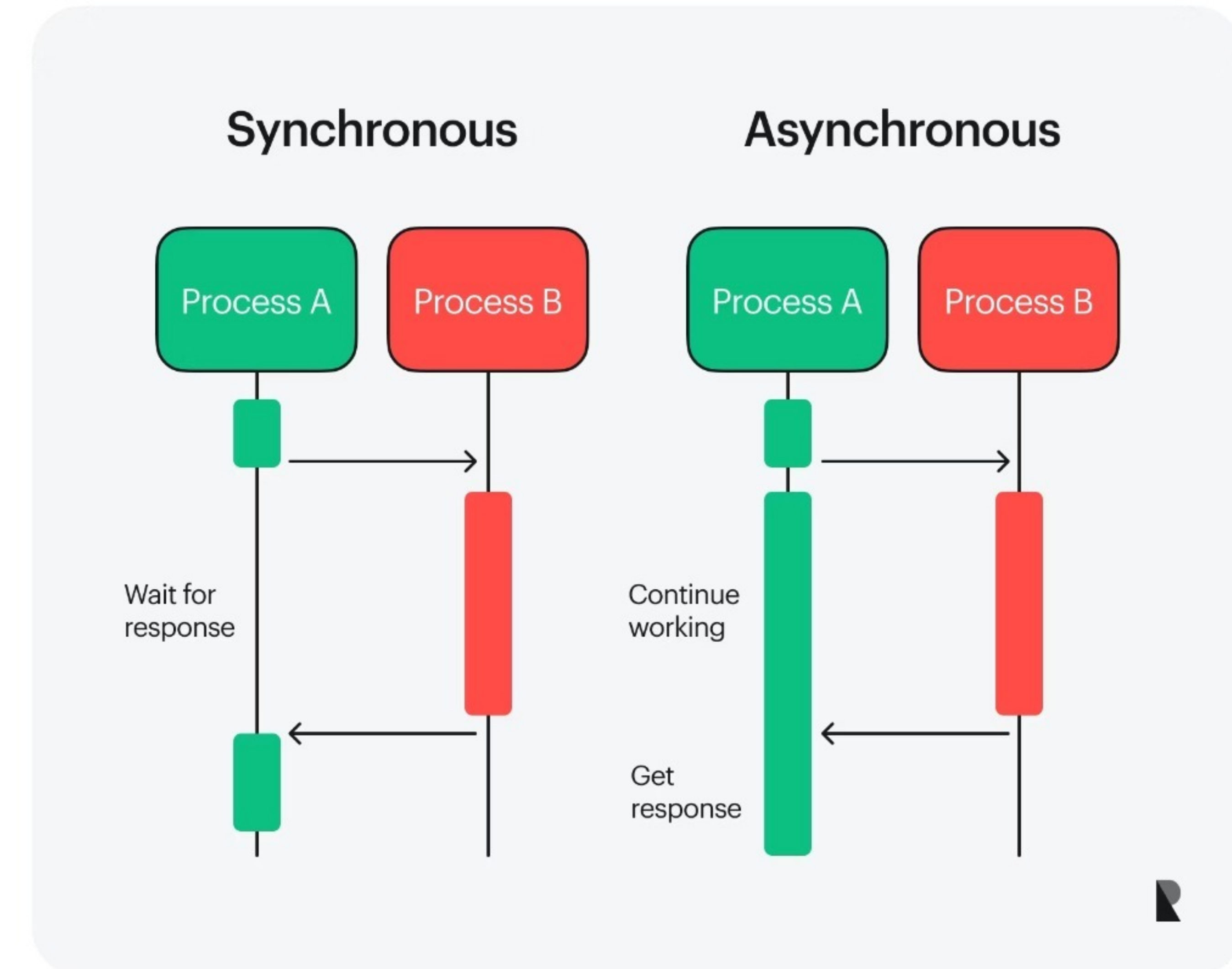
3.1 Synchronous Message

Line: solid.

Arrowhead: solid, filled triangle.

Meaning: the caller sends the message and blocks — it stops and waits until the receiver finishes and replies. Nothing else happens on the caller's lifeline during that wait.

Why solid + filled arrowhead specifically? UML reserves the filled arrowhead for "this is a real method call, and control transfers to the receiver until it responds." It's the default for almost every method call in a typical backend flow — `login()`, `getMovieById()`, `createBooking()` — because in a normal synchronous codebase (like a Spring Boot REST call chain), the calling thread genuinely does wait for the return value before moving to the next line of code.



3.2 Asynchronous Message

Line: solid.

Arrowhead: open/lined (a simple "V" shape, not filled).

Meaning: the caller sends the message and does not wait. It immediately continues its own work. The receiver processes the message independently, on its own time.

Why the different arrowhead? The open arrowhead is UML's way of saying "message sent, but no blocking relationship implied." This distinction exists because in real systems — message queues, event buses, fire-and-forget notifications — the caller's thread is free the instant the message is sent.

3.3 Return / Reply Message

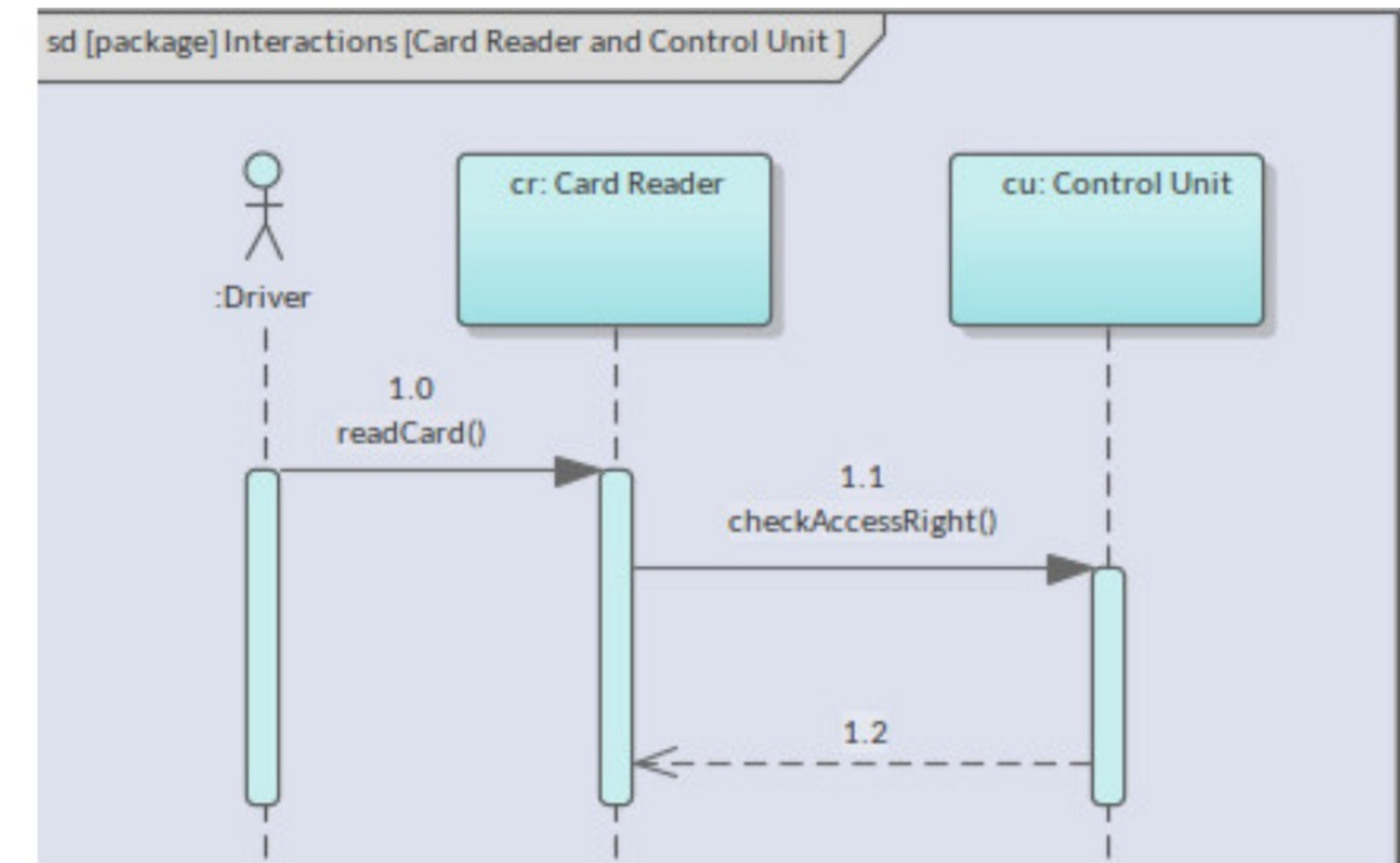
Line: dashed.

Arrowhead: open/lined, pointing back toward the original caller.

Meaning: this is the response to a synchronous call — the value (or void) being handed back.

Why dashed instead of solid? Solid lines represent an active request — an instruction to do something. A return isn't a new instruction; it's data flowing back passively as a consequence of the earlier call. UML draws it dashed specifically so your eye can instantly tell "this is a call" vs "this is a result," even in a busy diagram.

Do you always draw it? Yes, always draw the arrow. Whether you label it depends: if the return type matters for understanding (a User, an AuthToken, a List<Movie>), label it. If it's a trivial void with no informational value, you can leave it unlabeled or just write "void" — but for teaching/interview diagrams, label everything. Precision reads as competence.



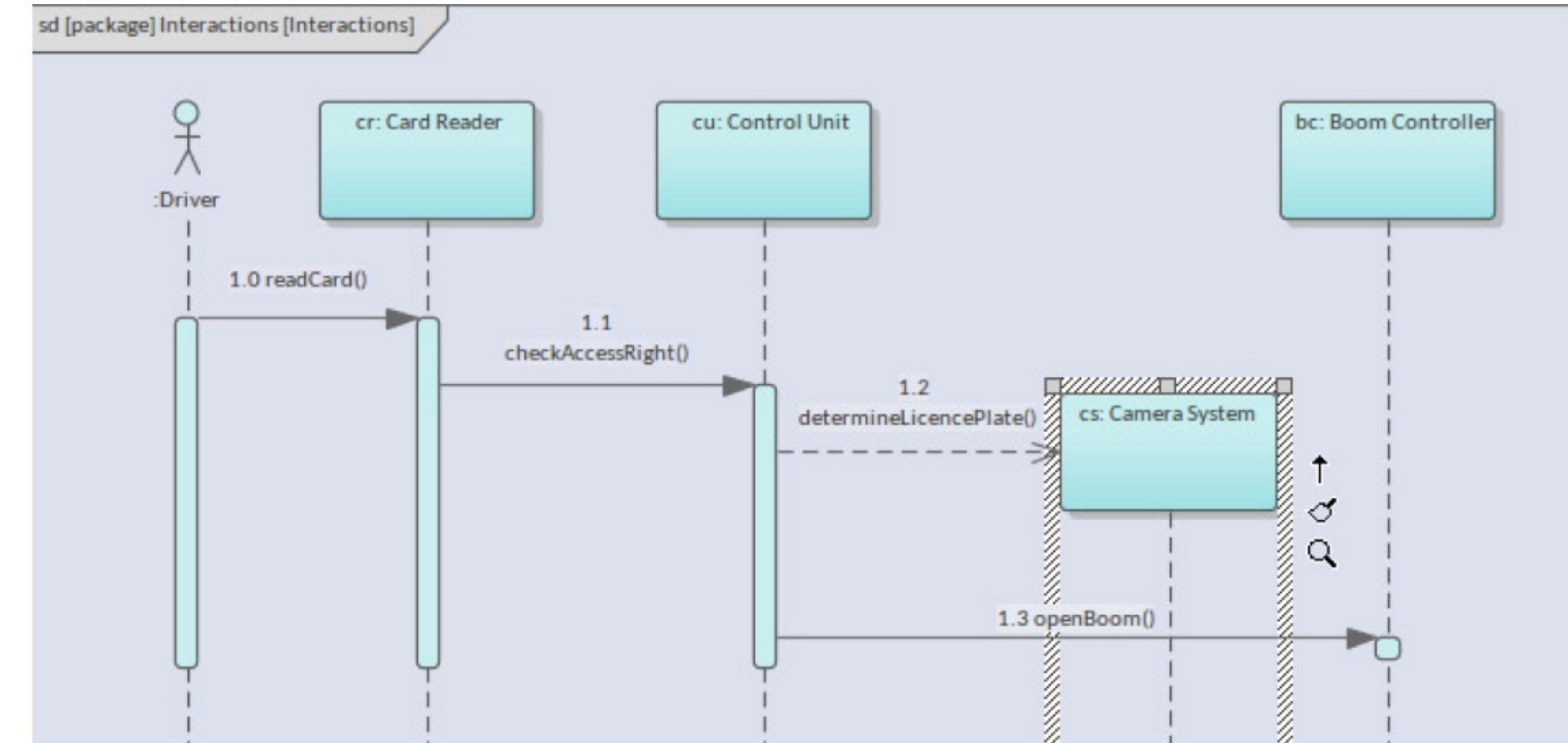
3.4 Create Message

Line: dashed.

Arrowhead: open, pointing to the top of a new lifeline (not to an existing box).

Meaning: this message is the moment, a brand-new object is instantiated. The receiving object's lifeline literally starts at the point this arrow lands — it does not exist before that moment.

Why dashed here, when it's clearly a "command" and not a return? UML treats "create" as its own distinct stereotype (<<create>>), and by convention it's drawn dashed to visually separate "this is bringing something into existence" from "this is a regular ongoing method call between two already-existing objects." Some notations do draw it solid with a <<create>> label instead — both are accepted, but dashed-with-label is the cleaner, more common convention and the one to default to.



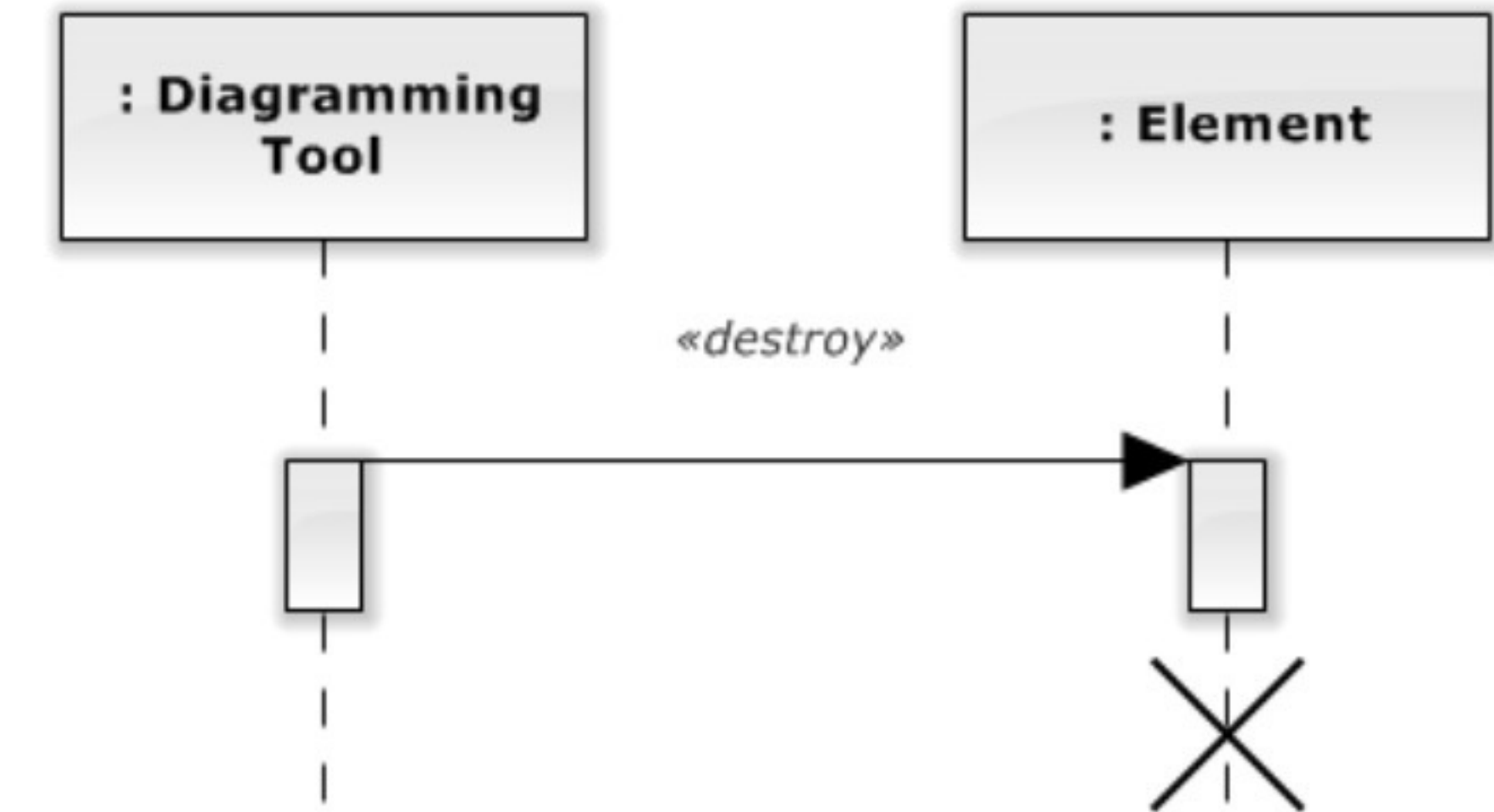
3.5 Delete / Destroy Message

Line: solid.

Arrowhead/ending: the message ends in a large X on the receiver's lifeline, and the lifeline stops there — it does not continue further down.

Meaning: the object's life is deliberately ended. It's no longer needed, and nothing later in the diagram may interact with it.

Why does the lifeline physically stop? Because after destruction, that object has no further existence in the system — so there's nothing left to draw. The X marks the exact moment of destruction; anything below it would be a logical error (you can't call a method on a destroyed object).

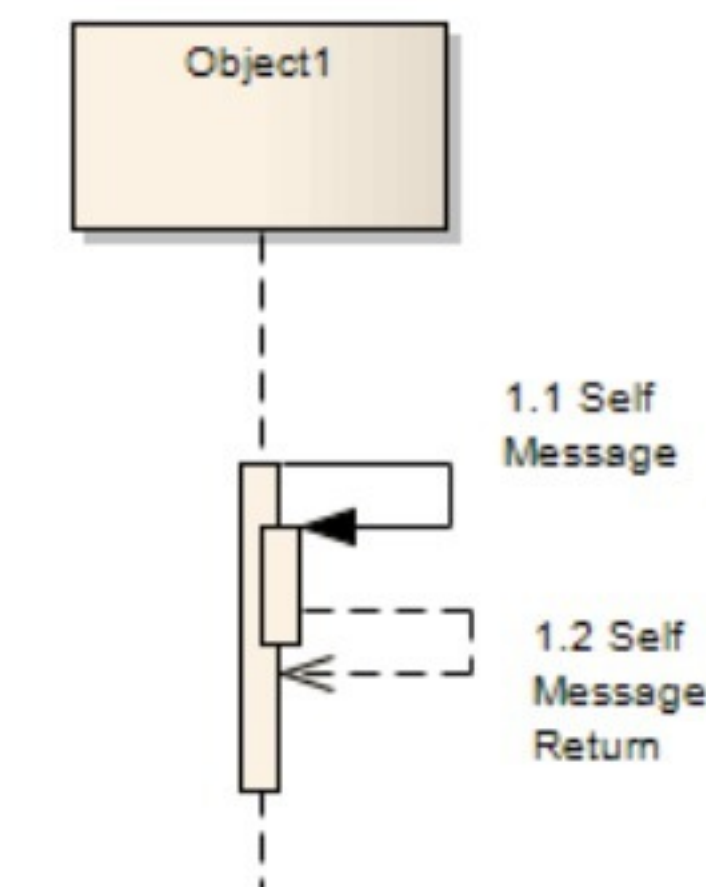


3.6 Self Message

Line: solid, drawn as a small loop that leaves the lifeline and comes back to the same lifeline.

Arrowhead: solid (it's still a synchronous call, just internal).

Meaning: an object calling one of its own methods — internal processing that doesn't involve any other participant.



Part 4 — Combined Fragments (Conditional / Repeated Flows)

Real flows aren't always a straight line. Two fragments you'll use constantly:

4.1 alt (Alternative)

Draw a rectangle around the messages, labelled alt in the top-left corner, with a condition in square brackets (e.g., [payment successful]). If there's a second possible path, split the box with a horizontal dashed line and label the second section with its own condition (e.g., [payment failed]) — this is the else equivalent.

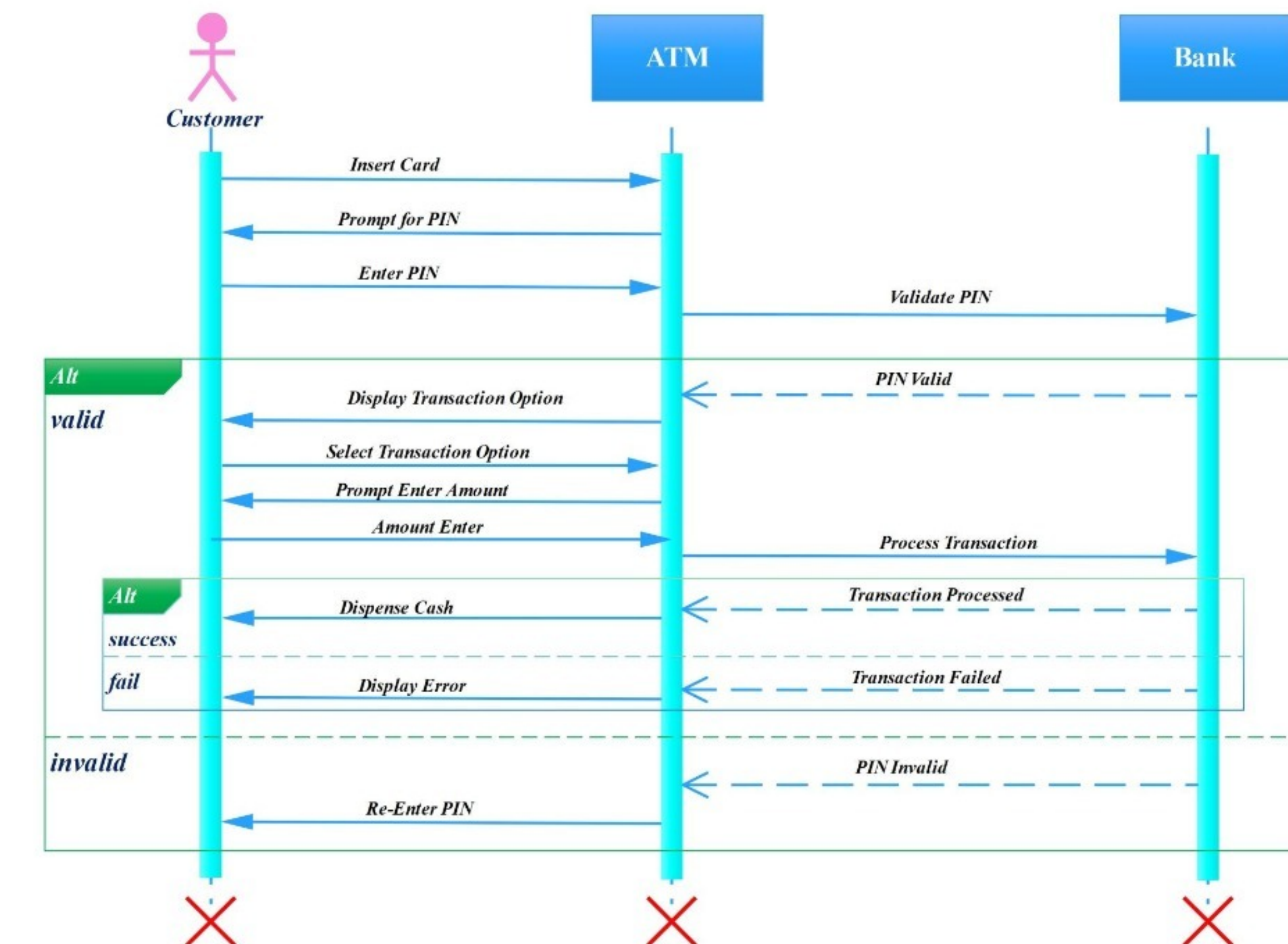
Why use this instead of two separate diagrams? Because the two outcomes share the same context up to the branch point. Splitting them into separate diagrams loses that shared setup and makes the diagram harder to follow as one coherent story. alt keeps the full decision visible in one place.

4.2 opt (Optional) — worth knowing even if not used today

Same rectangle style, labelled opt, single condition, no else-branch. Used when a step might happen but there's no alternative path if it doesn't — the flow just skips it.

4.3 loop — worth knowing even if not used today

Same rectangle style, labelled loop, with a condition or range (e.g., loop [for each seat in cart]). Used when a message sequence repeats.



Message Type	Line	Arrowhead	Used For
Synchronous call	Solid	Filled triangle	Normal method call, caller waits
Asynchronous call	Solid	Open/lined	Fire-and-forget, caller doesn't wait
Return/Reply	Dashed	Open, pointing back	Response to a synchronous call
Create	Dashed	Open, to top of new lifeline	Object instantiation
Delete/Destroy	Solid	Ends in X	Object explicitly destroyed
Self message	Solid loop	Filled triangle	Internal method call on same object

